```
--------------------------------------------------------------------------------¶-- TLALOC¬Strict Ada 83 Compiler¶-- Copyright (C) 2024-2026 Vincent M
ORIN - Universit de Bretagne Occidentale¶--¶-- This program is free software: you can redistribute it and/or modify¶-- it under the terms of the GNU G
eneral Public License as published by¶-- the Free Software Foundation, either¬version 3¬of the License, or¶-- (at your option)¬any later¬version.¶--¶-
- This program is distributed in the hope that it will be useful,¶-- but WITHOUT ANY WARRANTY; without even the implied warranty of¶-- MERCHANTABILITY
 or FITNESS¬FOR A PARTICULAR PURPOSE. See¬the¶-- GNU General Public License¬for more details.¶--¶-- You should have received a¬copy of the GNU General
 Public License¶-- along with this program. If not, see¬<https://www.gnu.org/licenses/>.¶----------------------------------------------------------------
-------------------¶¶------------------------------------------------------------------------------------------------------------¶--¬¬EXPA
NDER.ADB¬VINCENT MORIN¬21/6/2024¬¬UNIVERSITE DE BRETAGNE OCCIDENTALE¶---------------------------------------------------------------------------------
---------------------------------------¶--¬1¬2¬3¬4¬5¬6¬7¬8¬9¬0¬1¶¶¶with DIANA_NODE_ATTR_CLASS_NAMES, IDL, TEXT_IO;¶use  DIANA_NODE_ATTR_CLASS_NAMES, I
DL, TEXT_IO;¶¬¬¬¬¬¬--------¶¬¬¬procedure¬¬EXPANDER¶¬¬¬¬¬¬--------¶is¶¶  procedure DBGSTOP;¶¶¶¬¬¬¬¬-----¶¬package¬¬UTILS¶¬¬¬¬-----¶is¶¶  DEBUG¬¬¬¬: BOOLEAN¬
:= TRUE;¶¶  tab¬¬¬¬: CHARACTER¬renames ASCII.HT;¶¶  MAX_INSTR¬¬¬: constant¬¬:= 10_000;¬¬¬¬¬--| NB MAX D'INSTRUCTIONS¶  MAX_LABEL¬¬¬: constant¬¬:= 10_00
0;¬¬¬¬¬--| NB MAX D'ETIQUETTES DE SAUT¶  MAX_UNIT¬¬¬: constant¬¬:= 2**11-1;¬¬¬¬¬--| NB MAX D'UNITES¬PROGRAMME¶  MAX_LEVEL¬¬¬: constant¬¬:= 2**5-1;¬¬¬¬¬--
| NB MAX DE NIVEAUX D'IMBRICATION¶  MAX_OFFSET¬¬¬: constant¬¬:= 2**15-1;¬¬¬¬¬--| 32K¶¶  type LABEL_TYPE¬¬¬is new NATURAL¬¬range 0 .. MAX_LABEL;¬¬¬¬--| T
YPE ETIQUETTE¶  subtype¬UNIT_NUM¬¬¬is INTEGER¬¬range 0 .. MAX_UNIT;¶  subtype¬LEVEL_NUM¬¬¬is NATURAL¬¬range 0 .. MAX_LEVEL;¶  subtype¬OFFSET_VAL¬¬is I
NTEGER¬¬range -MAX_OFFSET .. MAX_OFFSET;¶¶  STORAGE_UNIT¬¬¬: constant¬¬:= 8;¬¬¬¬¬¬--| OCTET¬DE 8 bits¶  STACK_ELEMENT_SIZE¬¬: constant¬¬:= 8;¬¬¬¬¬¬--| L
A PILE EST GEREE PAR QUAD WORDS SUR X86-64¶  ADDR_SIZE¬¬¬: constant¬¬:= 8;¬¬¬¬¬¬--| ADRESSES SUR 64¬BITS¶  BOOL_SIZE¬¬¬: constant¬¬:= 1;¬¬¬¬¬¬--| BOOLEE
N SUR 1 OCTET¶  CHAR_SIZE¬¬¬: constant¬¬:= 1;¬¬¬¬¬¬--| CARACTERE SUR 8¬BITS¶  INTG_SIZE¬¬¬: constant¬¬:= 4;¬¬¬¬¬¬--| ENTIER SUR 32 BITS¶¶  type LOOP_COD
E¬¬¬is (¶¬¬DEC,   GT,    INC,¬ LT¬¬);¶¶  OUTPUT_CODE¬¬¬: BOOLEAN¬¬¬:= TRUE;¬¬¬¬¬¬-- Dans le traitement de spcif on dsactive le¬codage¶  IN_GENERIC_INST
ANTIATION¬¬: BOOLEAN¬¬¬:= FALSE;¬¬¬¬¬¬-- Traitement special pour les spec d instantiation¶  INSTANTIATION_MODEL_NAME¬¬: TREE;¶  GENERIC_MODEL_DECL_SEQ¬
¬: SEQ_TYPE;¶  IN_GENERIC_BODY¬¬¬: BOOLEAN¬¬¬:= FALSE;¬¬¬¬¬¬-- Traitement special pour les corps de¬generique¶¶  IN_SPEC_UNIT¬¬¬: BOOLEAN;¶¶  CUR_LEVEL
¬¬¬: LEVEL_NUM;¬¬¬¬¬¬¬¬--| NIVEAU D'IMBRICATION COURANT¶  CUR_OFFSET¬¬¬: OFFSET_VAL¬¬:= 0;¶¶  NO_SUBP_PARAMS¬¬¬: BOOLEAN¬¬¬:= TRUE;¬¬¬¬¬¬--| pour prms e
t prm_siz¶  ENCLOSING_BODY¬¬¬: TREE;¶  CHOICE_OTHERS_FLAG¬¬: BOOLEAN¬¬¬:= FALSE;¶¶  LOOP_STM_S¬¬¬: TREE;¶  LOOP_OP_INC_DEC¬¬¬: LOOP_CODE;¬¬¬¬¬¬¬¬--| PO
UR LE TRAITEMENT DES BOUCLES FOR REVERSE¶  LOOP_OP_GT_LT¬¬¬: LOOP_CODE;¬¬¬¬¬¬¬¬--| DE MEME¶¶  TYPE_SYMREP¬¬¬: TREE;¬¬¬¬¬¬¬¬¬--| UTILISE POUR LES OBJECT_
DECL VAR CONST¶¶¶¶  procedure OPEN_OUTPUT_FILE¬¬( FILE_NAME :STRING¬);¶  procedure CLOSE_OUTPUT_FILE;¶¶¶  function  OPER_SIZ_CHAR¬¬( DEFN :TREE )¬¬¬re
turn CHARACTER;¶  function  EXP_TYPE_CHAR¬¬( EXP :TREE )¬¬¬return CHARACTER;¶¶  function  NEW_LABEL¬¬¬¬¬¬return LABEL_TYPE;¶  function  NEW_LABEL¬¬¬¬¬
¬return STRING;¶  function  LABEL_STR¬¬( LBL : LABEL_TYPE )¬¬return STRING;¶¶  procedure INC_LEVEL;¶  procedure DEC_LEVEL;¶  function  CODE_DATA_TYPE_
OF¬¬( EXP_OR_TYPE_SPEC :TREE )¬¬return CHARACTER;¶  procedure LOAD_MEM¬¬( DEFN :TREE );¶  procedure STORE¬¬¬( DEST_DEFN :TREE );¶  function  TAB50¬¬¬¬
¬¬¬return STRING;¶¶  function  IMAGE¬¬¬( I : NATURAL )¬¬¬return STRING;¶¶  procedure REGIONS_PATH¬¬( ID : TREE; WITH_DOT :BOOLEAN := TRUE );¶¶¶  OPERA
ND_OVERFLOW¬¬¬: exception;¶¶¶end¬UTILS;¶¬-----¶¶  package¬CODI¬renames UTILS;¶  use CODI;¶¶  package¬body UTILS is separate;¶¶--  procedure CODE_ITERA
TION_ID¬¬( ITERATION_ID :TREE );¶¶  procedure CODE_ROOT ( ROOT :TREE );¶  procedure CODE_OBJECT ( OBJECT :TREE );¶  procedure CODE_SELECT_ALT_PRAGMA (
 SELECT_ALT_PRAGMA :TREE );¶  procedure CODE_EXCEPTION_ID¬( EXCEPTION_ID :TREE );¶  --|--------------------------------------------------------------
----------------------------¶¶¶¶¬¬¬¬¬-------------¶¬package¬¬¬EXPRESSIONS¶¬¬¬¬¬-------------¶  is¶¶    procedure CODE_EXP¬¬( EXP¬¬:TREE );¶    procedure COD
E_INDEXED¬¬( INDEXED¬¬:TREE );¶    procedure CODE_STRING_LITERAL¬( STRING_LITERAL¬:TREE; STR_NAME :STRING );¶    procedure CODE_SELECTED¬¬( SELECTED¬:
TREE; IS_SOURCE :BOOLEAN := TRUE );¶    procedure CODE_SLICE¬¬( SLICE¬¬:TREE; IS_DESTINATION :BOOLEAN := TRUE );¶¶¶  private¶¶    procedure CODE_NAME¬
¬( NAME : TREE );¶    procedure CODE_EXP_EXP¬¬( EXP_EXP¬:TREE );¶    procedure CODE_USED_OP¬¬( USED_OP¬¬:TREE );¶    procedure CODE_USED_NAME_ID¬( USE
D_NAME_ID¬:TREE );¶    procedure CODE_USED_CHAR¬¬( USED_CHAR :TREE );¶    procedure CODE_USED_OBJECT_ID¬( USED_OBJECT_ID¬:TREE );¶    procedure CODE_A
LL¬¬( ADA_ALL¬¬:TREE );¶¶    procedure CODE_ATTRIBUTE¬¬( ATTRIBUTE¬:TREE );¶    procedure CODE_FUNCTION_CALL¬( FUNCTION_CALL¬:TREE );¶    procedure CO
DE_QUALIFIED_ALLOCATOR¬( QUALIFIED_ALLOCATOR:TREE );¶    procedure CODE_SUBTYPE_ALLOCATOR¬( SUBTYPE_ALLOCATOR¬:TREE );¶    procedure CODE_AGGREGATE¬¬(
 AGGREGATE¬:TREE );¶¶    procedure CODE_NUMERIC_LITERAL¬( NUMERIC_LITERAL¬:TREE );¶    procedure CODE_NULL_ACCESS¬( NULL_ACCESS¬:TREE );¶    procedure
 CODE_SHORT_CIRCUIT¬( SHORT_CIRCUIT¬:TREE );¶    procedure CODE_PARENTHESIZED¬( PARENTHESIZED :TREE );¶    procedure CODE_CONVERSION¬¬( CONVERSION¬:TR
EE );¶    procedure CODE_QUALIFIED¬¬( QUALIFIED¬:TREE );¶    procedure CODE_RANGE_MEMBERSHIP¬( RANGE_MEMBERSHIP¬:TREE );¶    procedure CODE_TYPE_MEMBE
RSHIP¬( TYPE_MEMBERSHIP¬:TREE );¶¶    procedure CODE_VC_ID¬¬( CONSTANT_ID¬:TREE );¶¶¬-----------¶  end¬EXPRESSIONS;¶¬-----------¶¶¶¶¶¬¬¬¬¬-------------¶
¬package¬¬¬DECLARATIONS¶¬¬¬¬¬-------------¶  is¶¶    procedure CODE_DECL¬¬( DECL :TREE );¶    procedure CODE_DECL_S¬¬( DECL_S :TREE );¶    procedure COD
E_SUBPROG_ENTRY_DECL¬( SUBPROG_ENTRY_DECL :TREE );¶    procedure CODE_PACKAGE_DECL¬( PACKAGE_DECL :TREE );¶    procedure CODE_HEADER¬¬( HEADER :TREE )
;¶    procedure CODE_PACKAGE_SPEC¬( PACKAGE_SPEC :TREE );¶    procedure CODE_GENERIC_DECL¬( GENERIC_DECL :TREE );¶¶¶  private¶    procedure CODE_NULL_
COMP_DECL¬( NULL_COMP_DECL :TREE );¶¶¬¬¬-- TYPE DECLARATION¶¶    procedure CODE_TASK_DECL¬¬( TASK_DECL :TREE );¶    procedure CODE_UNIT_DECL¬¬( UNIT_D
ECL :TREE );¶    procedure CODE_SIMPLE_RENAME_DECL¬( SIMPLE_RENAME_DECL :TREE );¶¶¬¬¬-- SUBPROGRAM DECLARATION¶¶    procedure CODE_SUBP_ENTRY_HEADER¬(
 SUBP_ENTRY_HEADER¬:TREE );¶    procedure CODE_PARAM_S¬¬( PARAM_S¬:TREE; FOR_FUNCTION¬:BOOLEAN := FALSE );¶    procedure CODE_PARAM¬¬( PARAM :TREE );¶
    procedure CODE_IN¬¬( ADA_IN :TREE );¶    procedure CODE_IN_OUT¬¬( ADA_IN_OUT :TREE );¶    procedure CODE_OUT¬¬( ADA_OUT¬:TREE );¶¶¬¬¬-- VAR/CONST
DECLARATION¶¶    procedure CODE_VC_NAME¬¬( VC_NAME¬:TREE );¶    procedure CODE_ID_S_DECL¬¬( ID_S_DECL :TREE );¶    procedure CODE_EXCEPTION_DECL¬( EXC

EPTION_DECL :TREE );¶    procedure CODE_DEFERRED_CONSTANT_DECL ( DEFERRED_CONSTANT_DECL :TREE );¶    procedure CODE_EXP_DECL¬¬( EXP_DECL :TREE );¶
procedure CODE_NUMBER_DECL¬( NUMBER_DECL :TREE¬);¶    procedure CODE_OBJECT_DECL¬( OBJECT_DECL :TREE¬);¶¶    procedure CODE_ID_DECL¬¬( ID_DECL¬:TREE )
;¶¶¬-----------¶  end¬DECLARATIONS;¶¬-----------¶¶  package¬body DECLARATIONS is separate;¶¶¶¶¬¬¬¬¬-------------¶¬package¬¬¬INSTRUCTIONS¶¬¬¬¬---------
---¶  is¶¶    procedure CODE_STM_S¬¬( STM_S :TREE );¶    procedure CODE_STM¬¬( STM :TREE );¶    procedure CODE_PROCEDURE_CALL¬( PROCEDURE_CALL :TREE;
USED_NAME_ID : TREE );¶¶¶  private¶¶    procedure CODE_TEST_CLAUSE_ELEM_S¬( TEST_CLAUSE_ELEM_S :TREE; STM_END_LBL¬:STRING );¶    procedure CODE_COND_C
LAUSE¬( COND_CLAUSE :TREE; STM_END_LBL :STRING );¶    procedure CODE_STM_ELEM¬¬( STM_ELEM :TREE );¶    procedure CODE_STM_PRAGMA¬¬( STM_PRAGMA :TREE )
;¶    procedure CODE_LABELED¬¬( LABELED¬:TREE );¶    procedure CODE_NULL_STM¬¬( NULL_STM :TREE );¶    procedure CODE_STM_WITH_EXP¬( STM_WITH_EXP :TREE
 );¶    procedure CODE_STM_WITH_EXP_NAME¬( STM_WITH_EXP_NAME¬:TREE );¶    procedure CODE_STM_WITH_NAME¬( STM_WITH_NAME :TREE );¶    procedure CODE_CAL
L_STM¬¬( CALL_STM :TREE );¶    procedure CODE_BLOCK_LOOP¬¬( BLOCK_LOOP :TREE );¶    procedure CODE_LOOP¬¬( ADA_LOOP :TREE );¶    procedure CODE_ASSIGN
¬¬( ASSIGN :TREE );¶    procedure CODE_IF¬¬( ADA_IF :TREE );¶    procedure CODE_CASE¬¬( ADA_CASE :TREE );¶    procedure CODE_BLOCK¬¬( BLOCK :TREE );¶
    procedure CODE_EXIT¬¬( ADA_EXIT :TREE );¶    procedure CODE_RETURN¬¬( ADA_RETURN :TREE );¶    procedure CODE_GOTO¬¬( ADA_GOTO :TREE );¶    procedur
e CODE_ACCEPT¬¬( ADA_ACCEPT :TREE );¶    procedure CODE_DELAY¬¬( ADA_DELAY :TREE );¶    procedure CODE_SELECTIVE_WAIT¬( SELECTIVE_WAIT :TREE );¶    pr
ocedure CODE_TERMINATE¬¬( ADA_TERMINATE :TREE );¶    procedure CODE_ENTRY_STM¬¬( ENTRY_STM :TREE );¶    procedure CODE_COND_ENTRY¬¬( COND_ENTRY :TREE
);¶    procedure CODE_TIMED_ENTRY¬( TIMED_ENTRY :TREE¬);¶    procedure CODE_ABORT¬¬( ADA_ABORT :TREE );¶    procedure CODE_CLAUSES_STM¬( CLAUSES_STM :
TREE¬);¶    procedure CODE_RAISE¬¬( ADA_RAISE :TREE );¶    procedure CODE_CODE¬¬( CODE :TREE );¶¶¬-----------¶  end¬INSTRUCTIONS;¶¬-----------¶¶  pa
ckage¬body EXPRESSIONS  is separate;¶¶¶¶¶¬¬¬¬¬-------------¶¬package¬¬¬STRUCTURES¶¬¬¬¬¬-------------¶  is¶¶    procedure CODE_COMPILATION_UNIT¬( COMPILATION_U
NIT :TREE );¶    procedure CODE_BLOCK_BODY¬¬( BLOCK_BODY :TREE );¶¶¶  private¶¶    procedure CODE_WITH_CONTEXT¬( CONTEXT_ELEM_S  :TREE );¶    procedur
e CODE_SUBPROGRAM_BODY¬( SUBPROGRAM_BODY :TREE );¶    procedure CODE_PACKAGE_BODY¬( PACKAGE_BODY :TREE );¶    procedure CODE_SUBUNIT_BODY¬( SUBUNIT_BO
DY :TREE );¶    procedure CODE_TASK_BODY¬¬( TASK_BODY :TREE );¶    procedure CODE_EXCEPTIONS_ALTERNATIVE_S ( ALTERNATIVE_S¬:TREE );¶¶¬----------¶  end
¬STRUCTURES;¶¬----------¶¶  package¬body STRUCTURES    is separate;¶  package¬body INSTRUCTIONS  is separate;¶¶¶¶¬¬¬¬¬----------¶  procedure¬¬¬CODE_ROOT
¬¬¬( ROOT :TREE )¶  is¶    USER_ROOT¬:constant¬TREE¬:= D( XD_USER_ROOT,¬ROOT );¶    COMPILATION¬:constant¬TREE¬:= D( XD_STRUCTURE,¬USER_ROOT¬);¶    CO
MPLTN_UNIT_S¬:constant¬TREE¬:= D( AS_COMPLTN_UNIT_S, COMPILATION );¶  begin¶    declare¶      COMPLTN_UNIT_SEQ¬: SEQ_TYPE¬:= LIST (¬COMPLTN_UNIT_S );¶
      COMPLTN_UNIT¬¬: TREE;¶    begin¶      while not IS_EMPTY( COMPLTN_UNIT_SEQ ) loop¶        POP( COMPLTN_UNIT_SEQ, COMPLTN_UNIT );¶        CODI.OP
EN_OUTPUT_FILE( GET_LIB_PREFIX & PRINT_NAME(¬D( XD_LIB_NAME, COMPLTN_UNIT ) ) );¶¶        STRUCTURES.CODE_COMPILATION_UNIT ( COMPLTN_UNIT );¶¶
CODI.CLOSE_OUTPUT_FILE;¶      end¬loop;¶    end;¶¶  end¬CODE_ROOT;¶¬---------¶¶¶¶  procedure CODE_CONTEXT_PRAGMA ( CONTEXT_PRAGMA :TREE ) is¶  begin¶
   null;¶  end;¶¶¶¶  procedure CODE_BLOCK_MASTER¬( BLOCK_MASTER :TREE ) is¶  begin¶    null;¶  end;¶¶¶¶  procedure CODE_DERIVED_SUBPROG ( DERIVED_SUBP
ROG :TREE ) is¶  begin¶    null;¶  end;¶¶¶¶  procedure CODE_IMPLICIT_NOT_EQ ( IMPLICIT_NOT_EQ :TREE ) is¶  begin¶    null;¶  end;¶¶¶¬¬¬¬¬--------------
--------¶  procedure¬¬CODE_SUBTYPE_INDICATION¬( SUBTYPE_INDICATION, TYPE_DECL :TREE )¶  is¬¬¬------------------------¶    TYPE_ID¬¬: TREE¬¬:= D( AS_SOU
RCE_NAME, TYPE_DECL );¶    INTEGER_SPEC¬¬: TREE¬¬:= D( SM_TYPE_SPEC,¬TYPE_ID );¶  begin¶    DI( CD_LEVEL,¬  INTEGER_SPEC, INTEGER( CODI.CUR_LEVEL ) );
¶    DB( CD_COMPILED,  INTEGER_SPEC, TRUE );¶¶  end¬CODE_SUBTYPE_INDICATION;¶¬-----------------------¶¶¶¶  procedure CODE_OBJECT ( OBJECT :TREE ) is¶
 begin¶    case OBJECT.TY is¶    when DN_VARIABLE_ID =>¶      PUT_LINE( tab¬& "La " &¬INTEGER'IMAGE( DI( CD_LEVEL, OBJECT ) )¬& ',' & tab & PRINT_NAME
( D( LX_SYMREP,¬OBJECT ) ) & "_disp" );¶¶    when DN_IN_ID =>¶      PUT_LINE( tab¬& "LVA " & INTEGER'IMAGE( DI(¬CD_LEVEL,¬OBJECT ) ) & ',' & tab & PRI
NT_NAME( D(¬LX_SYMREP, OBJECT )¬) );¶¶    when DN_IN_OUT_ID | DN_OUT_ID =>¶      PUT_LINE( tab¬& "LVA " & INTEGER'IMAGE( DI(¬CD_LEVEL,¬OBJECT ) ) & ',
' & tab & PRINT_NAME( D(¬LX_SYMREP, OBJECT )¬) );¶¶    when DN_INDEXED¬=>¶      EXPRESSIONS.CODE_INDEXED( OBJECT );¶¶    when DN_USED_OBJECT_ID =>¶
  CODE_OBJECT( D( SM_DEFN, OBJECT )¬);¶¶    when DN_CONSTANT_ID =>¶      PUT_LINE( tab¬& "LIa " & INTEGER'IMAGE( DI(¬CD_LEVEL,¬OBJECT ) ) & ','¶¬
  & tab & PRINT_NAME( D( LX_SYMREP,¬OBJECT ) ) & "_disp" );¬¬¬¬¬¬-- LOAD CONSTANT ADDRESS¶¶    when others =>¶      PUT_LINE( "!!! LOAD_OBJECT_ADDRESS
: OBJECT.TY ILLICITE " & NODE_NAME'IMAGE ( OBJECT.TY ) );¶      raise PROGRAM_ERROR;¶    end case;¶  end;¶¶¶¶  procedure CODE_ADRESSE ( ADRESSE :TREE
) is¶  begin¶    case ADRESSE.TY¬is¶    when DN_VARIABLE_ID =>¶null;--¬    GEN_PUSH_DATA ( A, DI (CD_COMP_UNIT,¬ADRESSE ) , LEVEL_NUM(DI ( CD_LEVEL, ADR
ESSE )), DI ( CD_OFFSET, ADRESSE ) );¶    when DN_IN_ID =>¶null;--¬   GEN_PUSH_DATA ( A, 0,  LEVEL_NUM(DI ( CD_LEVEL, ADRESSE )), DI ( CD_OFFSET, ADRE
SSE ) );¶    when DN_IN_OUT_ID | DN_OUT_ID =>¶null;--¬   GEN_PUSH_DATA ( A, 0, LEVEL_NUM(DI( CD_LEVEL, ADRESSE )), DI( CD_VAL_OFFSET,¬ADRESSE )¬);¶
 when DN_INDEXED¬=>¶      EXPRESSIONS.CODE_INDEXED ( ADRESSE );¶    when DN_USED_OBJECT_ID =>¶      CODE_ADRESSE ( D( SM_DEFN, ADRESSE ) );¶    when o
thers =>¶    PUT_LINE ( "!!! CODE_ADRESSE : OBJECT.TY ILLICITE " & NODE_NAME'IMAGE ( ADRESSE.TY ) );¶      raise PROGRAM_ERROR;¶    end case;¶  end;¶¶
¶¶  procedure CODE_SELECT_ALT_PRAGMA ( SELECT_ALT_PRAGMA :TREE ) is¶  begin¶    null;¶  end;¶¶¶¶  procedure CODE_EXCEPTION_ID¬( EXCEPTION_ID :TREE ) i
s¶  begin¶    declare¶      LBL¬:constant¬STRING :=¬NEW_LABEL;¶    begin¶--      DI ( CD_LABEL, EXCEPTION_ID, INTEGER ( LBL ) );¶PUT_LINE(¬"; EXL" &¬t
ab & LBL¬);¶--      EMIT ( EXL,¬LBL, S=> PRINT_NAME¬( D ( LX_SYMREP, EXCEPTION_ID¬) ),¶--¬      COMMENT=> "NUMERO D EXCEPTION SUR DECLARATION" );¶    e
nd;¶  end;¶¶¶¶  procedure DBGSTOP¬is begin null; end;¶¶¶begin¶  OPEN_IDL_TREE_FILE( LIB_PATH(1..LIB_PATH_LENGTH) & "$$$.TMP" );¶  if DI( XD_ERR_COUNT,
 TREE_ROOT ) = 0¶  then¶    CODE_ROOT( TREE_ROOT );¶  end if;¶  CLOSE_IDL_TREE_FILE;¶¶¬---------¶end¬EXPANDER;¶¬---------¶-----------------------------
-------------------------------------------------------------------------------------¶--¬1¬2¬3¬4¬5¬6¬7¬8¬9¬0¬1¬2¶
```